RoboFlow

Guia simples para entender o projeto e apresentar sem complicação

1. A ideia principal

Imagine um carrinho industrial em uma fábrica. Ele segue uma linha no chão e precisa tomar decisões quando chega a lugares como carga, descarga ou cruzamento.

Esse carrinho é chamado de AGV: veículo guiado automaticamente. O RoboFlow é a linguagem criada para dizer o que ele faz em cada estação.

> Exemplo: ao chegar à estação de carga, o robô deve parar, esperar a carga terminar e depois voltar a seguir a linha.

```
station "carga" {
    on_arrival {
        stop()
        wait_signal("carga_completa", timeout: 30s)
        resume_line_following()
    }
}
```

Traduzindo o código:

- stop(): o robô para.
- wait_signal(...): ele espera um sinal externo.
- timeout: 30s: ele espera no máximo 30 segundos.
- resume_line_following(): ele volta a seguir a linha.

2. Por que criar uma linguagem?

RoboFlow é uma DSL, ou Linguagem de Domínio Específico. Ela foi criada para resolver um problema específico: definir o comportamento do AGV nas estações.

Parte do sistema	Responsabilidade
ESP32	Lê sensores, controla motores, segue a linha com PID e mantém a segurança.
RoboFlow	Descreve decisões: parar, esperar, virar, sinalizar e voltar a seguir a linha.

O RoboFlow não ensina o robô a seguir uma linha. Essa parte é controle contínuo e fica no firmware. A linguagem descreve a lógica de decisão: “cheguei na estação X, então faça Y”.

3. O que é um compilador?

Um compilador é um programa que lê outro programa e tenta entendê-lo. Ele trabalha por etapas:

```
Arquivo RoboFlow (.rf)
        ↓
Análise léxica → Tokens
        ↓
Análise sintática → AST
        ↓
Futuro: análise semântica → geração de código → robô
```

> No estado atual do projeto, as três primeiras partes já existem: lexer, parser e AST. A validação semântica e a geração de código ainda são próximas etapas.

4. Análise léxica: separar as peças

A análise léxica é a primeira leitura do código. Ela pega os caracteres e os separa em unidades chamadas tokens. Pense nela como alguém separando uma frase em palavras, números e símbolos.

```
wait_signal("ok", timeout: 30s)

KEYWORD(wait_signal)  LPAREN  STRING(ok)  COMMA
KEYWORD(timeout)  COLON  DURATION(30.0)  RPAREN
```

Exemplo	Tipo de token
station, stop, if	Palavra reservada
"carga"	String
0.15	Número
30s	Duração
{ } ( ) , : ==	Símbolos e operador

O lexer ignora espaços e comentários que começam com `//`. Também aponta erros simples, por exemplo `stop(@)`: o símbolo `@` não pertence à linguagem.

Frase para decorar:

> “A análise léxica lê o código e o divide em tokens, como palavras reservadas, números, strings e símbolos.”

5. Análise sintática: verificar a estrutura

Depois que o lexer separa os tokens, o parser verifica se eles estão organizados corretamente. Essa organização é definida pela gramática da linguagem.

Uma estação precisa seguir uma forma parecida com esta:

```
station "nome" {
    on_arrival {
        comandos
    }
}
```

Se faltar uma chave ou um parêntese, a estrutura fica errada. Por exemplo:

```
station "carga" {
    on_arrival {
        stop()
}
```

Aqui falta uma chave de fechamento. Isso é um erro sintático.

Frase para decorar:

> “A análise sintática verifica se os tokens obedecem à gramática da linguagem.”

6. O que é a AST?

AST significa Árvore de Sintaxe Abstrata. Depois de verificar que o código está bem estruturado, o parser cria uma árvore que guarda a lógica do programa.

```
PROGRAMA
└── ESTAÇÃO "carga"
    ├── stop()
    ├── wait_signal("carga_completa", 30s)
    └── resume_line_following()
```

A AST elimina detalhes como chaves e parênteses e mantém o que importa: estação, comandos, argumentos e condições. Ela será usada nas próximas fases do compilador.

7. Condicionais

O projeto também entende decisões do tipo “se... então... senão...”:

```
if next_destination == "linha_2" {
    turn_right()
} else {
    continue_straight()
}
```

Isso significa: se o próximo destino for linha 2, vire à direita; caso contrário, siga em frente.

8. Os três tipos de erro

Tipo	Exemplo	Explicação simples
Léxico	stop(@)	Há um símbolo que não pertence à linguagem.
Sintático	Falta uma }	As peças existem, mas estão na estrutura errada.
Semântico	set_speed(999999)	Está bem escrito, mas não faz sentido para o robô.

A análise semântica ainda não está pronta. No futuro ela deve verificar, por exemplo, se existe uma única estação `default`, se a velocidade é possível e se cada caminho termina com segurança.

A frase mais importante do trabalho

> “O lexer verifica os elementos da linguagem, o parser verifica a estrutura e a análise semântica verifica se o programa faz sentido para o AGV.”

9. Roteiro de fala para sua apresentação

Você pode falar praticamente assim:

“Nosso projeto é uma linguagem chamada RoboFlow para definir o comportamento de um AGV, um carrinho industrial seguidor de linha. O ESP32 fica responsável por sensores, motores, PID e segurança. A nossa linguagem define o que o robô faz quando chega a uma estação, como parar, esperar, virar e voltar a seguir a linha.”

“Primeiro, o compilador faz a análise léxica e separa o código em tokens. Depois, faz a análise sintática e verifica se esses tokens estão na estrutura correta. Quando está tudo certo, o parser gera uma AST, que é uma árvore representando a lógica do programa.”

“Hoje já funcionam análise léxica, análise sintática e AST. Ainda faltam análise semântica, geração de código e integração com o ESP32.”

Se o professor perguntar “o que já funciona?”, responda:

> “Já funciona o front-end do compilador: lexer, parser e AST. As próximas etapas são validação semântica, geração de código e integração embarcada.”